

# **REPORT**

## **Executive Summary**

This report encapsulates the design, implementation, and simulation of a microprocessor unit in VHDL which computes the total and average cost of items in a shopping basket. The project involves the creation and subsequent elevation of a basic arithmetic logic unit (ALU) into an autonomous system which has the capacity to fetch data and instructions from separate entities, compute values based on the instructions, and store the results in a specified address. The initial ALU was enhanced with the addition of programs to automate instruction sequencing. This included the creation of a new memory entity containing specified instructions for the process. The system was tested and verified through simulations which demonstrated the system's ability to fetch data from the memory, sum all the values with each other, and successfully produce the average cost of the shopping basket based on the specified data values provided in the ROM memory.

## **Introduction**

The main purpose of this assignment was to design and implement a microprocessor unit which is adept in the execution of instructions from a memory entity to complete a certain task. This project is a culmination of fundamental concepts incrementally implemented from previous assignments such as memory elements, an arithmetic logic unit, and an automated microprocessor system.

The ALU and a data memory entity (ROM) was initialised during the laboratory sessions to provide platforms for computation and storage of data. These components relied on the operation of switches to function. These were then enhanced into a more autonomous system through the use of a fetch instruction which synchronised the process with the defined clock efficiently replacing the use of switches.

The design was used to calculate the sum and subsequent average cost of products in a shopping basket. The average was determined through the application of arithmetic shifting in the ALU and a new instruction-based memory entity. This report captures the overall architecture of the system as well as the simulation results confirming its functionality.

## Design

The microprocessor design is in the form of a Harvard-style architecture meaning the memory is divided into two sections of instruction and data memory. The VHDL entities which are included in this design are the Top-level controller (top-level), arithmetic logic unit (alu), instruction memory (instructions), data memory (rom), and a testbench (tb).

### 1. General Architecture:

The control logic is centred around a program counter defined as 'program' in the Toplevel entity. This replaced the use for switches in the initial version of the program hence, allowing the system to be synchronised with the 100 MHz clock. The instruction memory delivers separate 16-bit instructions based on the address received from the program counter. The instruction has 4-bit subsections with the first four bits being the opcode, next four being the destination address, then the two data addresses make up the last eight bits.

```
--setting up program counter
process (CLK100MHZ)
begin
    if rising_edge (CLK100MHZ) then

        program <= program + 1;

    end if;
end process;
```

**Figure 1. Initialization of the Program Counter**

```
my_instructions: entity work.instructions(Behavioral) port map(address => program, instruction => present_instruction);
```

**Figure 2. Instantiation of instruction memory into top-level file**

## 2. Arithmetic Logic Unit (ALU):

The ALU is the computational engine. The entity contains the mathematical operations and assigns them to a unique opcode which can be called upon to perform their corresponding operation. It was initially designed to be used with 8-bit data types and perform simple operations like addition, subtraction, and bitwise logic. It was then enhanced to work with 32-bit data types to accommodate for the computation of the average cost of items without misrepresenting the data. The following opcodes were allocated to each operation:

**Table 1. Opcodes and their allocated operations**

| Last Digit of ID number | 0  |
|-------------------------|----|
| num1+num2               | 10 |
| num1-num2               | 8  |
| num2-num1               | 6  |
| num1                    | 14 |
| num1 AND num2           | 2  |
| num1 OR num2            | 13 |
| num1 XOR num2           | 15 |
| shift right num1 arith  | 11 |
| shift right num1 log    | 3  |

The opcodes were converted to binary before being implemented as VHDL code into the system.

## 3. Instruction Memory(instructions):

This entity contains the instructions called upon to calculate the total and subsequent average of the price of items in the shopping basket. The addresses 1 through 8 are iterated through by the program, summed together, and stored in a specific location(0000). The calculation of the average is done by division through three consecutive 'shift right arithmetic' operations on the result of the addition. This performs a division by two, three times hence dividing the total by eight which is the total number of items.

## 4. Data memory(rom):

The values of data were initialised in this entity. The data values were arranged in an array allowing for simple addressing. Addresses and data outputs were set up in this entity as well as a link between the data and another input address creating a pathway for a loop. The data values were initially converted to 8-bit hexadecimal figures in pounds but eventually enhanced to be represented as 32-bit figures in pence to accommodate for finding the average.

**Table 2. Data Values**

| Second-from-last digit of ID number: | 3 (values in pence) |
|--------------------------------------|---------------------|
| 0                                    | 0                   |
| 1                                    | 600                 |
| 2                                    | -1500               |
| 3                                    | 1800                |
| 4                                    | 2200                |
| 5                                    | 2400                |
| 6                                    | -2600               |
| 7                                    | 2700                |
| 8                                    | 1900                |
| 9-15                                 | 0                   |

```

BEGIN
PROCESS (clk)
BEGIN
IF rising_edge(clk) then
rom_data( TO_INTEGER (address3)) <= data3;
END IF;
END PROCESS;

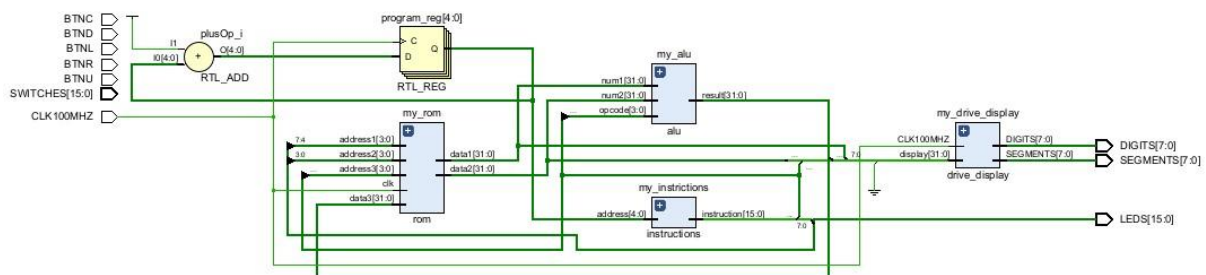
data1 <= rom_data ( TO_INTEGER (address1) );
data2 <= rom_data ( TO_INTEGER (address2) );

```

**Figure 3. Creation of data outputs and addresses**

Testbench:

Previously, the testbench entity contained the iterative instructions for controlling the switches to toggle to specific opcodes and memory values. Since the operation of the microprocessor is no longer centres around the toggling of switches, the testbench only retained its initialisation of the 100 MHz clock and a wait command with a specified time allowing the program to complete without interruptions.



**Figure 4. Block diagram of the complete system.**

## Results

To ensure the proper functionality of the microprocessor, a behavioral simulation of the VHDL code was carried out using the Vivado software simulator. The simulation provides a visual representation of how the microprocessor functions at specific time intervals synchronised to the rising edge of the clock.

The figures below show the process of the microprocessor iterating through the program and the subsequent values associated at each point.

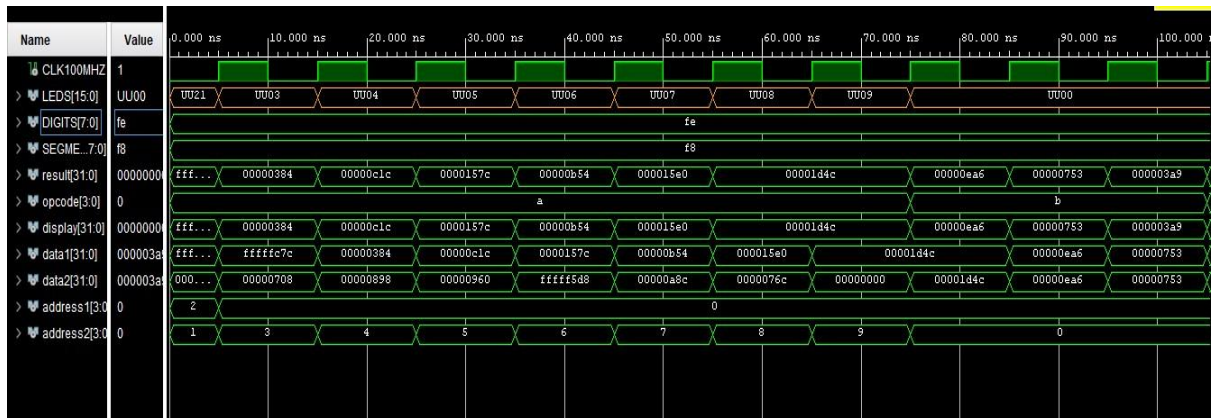
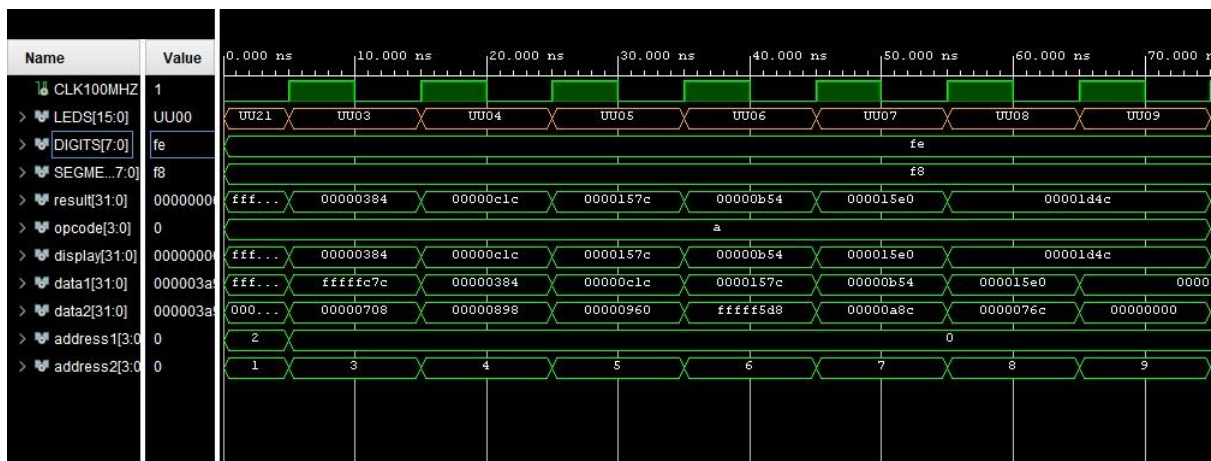
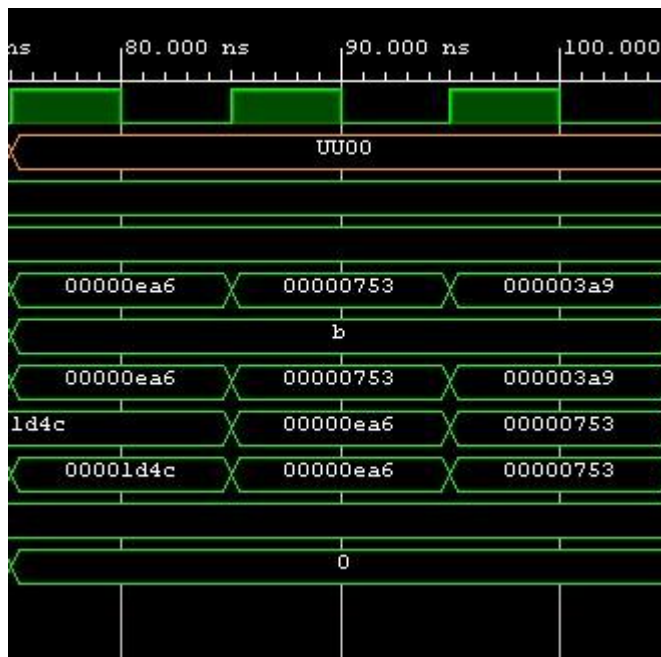


Figure 5. Full simulation



**Figure 6. Addition section (opcode = 10)**



**Figure 7. Shift right section (opcode = 11)**

Figure 4 above shows the steady incrementation of the signal for 10 complete clock cycles. The address signal lines show where the system is fetching the data values from the rom. It initially starts with address 1 and 2 selecting the first and second elements from the data memory then from the next clock cycle, only address 2 iterates for the remaining values to ensure they all get summed up to each other.

The data signal lines represent the values corresponding to their locations referred to by the address signal. The initial values are shown to be the first and second values in the data memory. However, after the first rising edge of the clock, the data 2 signal lines continues to iterate through the rest of the rom data but the data 1 signal line returns the sum of the previous two values due to the loop that was set up.

The result is split into two sections. Figure 5 above shows the simulation up until the point where the opcode changes from addition (10) to shift-right (11). The result signal line up until that point shows the addition of the data 1 and data 2 values at any given time. By the sixth risen edge of the clock, the result displayed the total of all the items in the shopping basket. This is verified by summing the data memory values together:

$$0 + 600 - 1500 + 1800 + 2200 + 2400 - 2600 + 2700 + 1900 = 7500$$

The result after the sixth clock cycle was 00001D4C<sub>16</sub>. Converting to decimal:

$$1 \times 16^3 + 13 \times 16^2 + 4 \times 16^1 + 12 \times 16^0 = 7500_{10}$$

This confirms that the program works as expected up to that point.

Figure 6 above shows the second section of the result when the opcode makes the system perform the shift-right operation (11). The simulation shows that this operation was performed on the final sum of the items three times to obtain the average cost of items.

The proper functionality of this part of the program was verified by calculating the average price of the items:

$$average = total \div number\ of\ items$$

$$\therefore average = 7500 \div 8 = 937.5 \text{ Then}$$

converting the final result of  $000003A9_{16}$  to decimal:

$$3 \times 16^2 + 10 \times 16^1 + 9 \times 16^0 = 937$$

The decimal value was truncated due to the calculation process being integer arithmetic.

The display outputs the result value at each point.

## **Evaluation and Conclusions**

### **Strengths**

The results shown in this report represents proper functionality of the designed microprocessor based on its requirements of being able to determine total and average of the items in a shopping basket. A major strength of the system lied in its enhancement from an instruction system relying of switches to an automated instruction execution system.

Moreover, the functionality of the system was further improved through the extension of the general architecture to make use of 32-bit signed numbers. This greatly improved the accuracy of the system as the initial 8-bit architecture wasn't entirely reliable due to its tendency to truncate fractional information during calculations.

The ability for the system to return not just the total value of items as defined in the initial design, but the provision of additional information being the average price of items gave the system an important layer of depth which could be useful in comparing different situations with each other.

### **Limitations**

Although the designs fulfilment of requirements, certain weaknesses could be pinpointed. For example, the ALU's operation in integer arithmetic slightly deterred the accuracy of calculations performed by the system. Due to the truncation of remainders by division through the bit shift operation, the actual average of 937.5 was presented as 937. This poses a problem in calculations concerning finances where decimal accuracy is required.

Furthermore, the program counter used only incremented by 1 each time until the program ended without being able to stray from this pathway. With these specifications, operations such as 'jump' were not possible. Hence, the overall operation of the system could be slowed down when dealing with a larger data set considering the program would have to start from the beginning and increment gradually until it reaches the required point.

## **Improvements**

The following modifications could be implemented into the system to overcome the above limitations.

The accuracy of the calculations could be improved with the implementation of a floating-point support in the ALU to allow for decimal value arithmetic. The inclusion of a register which would be used to store the remainders could also be useful in storing the omitted information from the shift-right operations performed by the microprocessor.

Also, a 'jump' instruction could be implemented into the system. This would allow the program to move between specific iterations without having to start from the beginning and sequentially step through the program before reaching that point. This would be especially helpful when dealing with larger sets of data.